

DRAGON BALL

Game Design & Engineering Blueprint

A research-grounded plan for building a Dragon Ball game that fuses **Sparking! ZERO-style 3D arena combat**, **Dragon Block Noea / DragonMineZ RPG progression**, and **canon-accurate power levels** into a single coherent title.

Synthesizes four research inputs:

- 1. Dragon Ball: Sparking! ZERO — Combat & Controls Guide (7 pp)*
- 2. Dragon Block Noea — Deep Research Guide (9 pp)*
- 3. Dragon Ball Z — Fact-Checked Power Levels (12 pp)*
- 4. Independent web research on fighting-game engine design, rollback netcode, animation state machines, voxel terrain*

TITLE BLOCK

Project: Dragon Ball Game
Scale: N/A (design doc)
Sheet: 01 of 01
Drawn: Z.ai Research
Date: 2026
Rev: A — Final
Status: RESEARCH BLUEPRINT

Table of Contents

1. Executive Summary — The Game Concept	4
1.1 The Three Pillars	4
1.2 The One-Sentence Pitch	4
2. The Three Source Systems — What Each Teaches	5
2.1 Comparative Matrix	5
2.2 What "Accurate Power Levels" Means in Practice	5
3. Why Sparking! ZERO's Combat Is So Effective	6
3.1 Reason 1 — Movement is the backbone, not combos	6
3.2 Reason 2 — Ki is the universal economy	6
3.3 Reason 3 — Layered counters with different costs	6
3.4 Reason 4 — Sparking! Mode is the shared climax	6
3.5 Reason 5 — Every character is a "setup archetype"	7
4. The Combat Loop — Anatomy of One Exchange	8
4.1 The Five-Phase Loop	8
4.2 Where the RPG Layer Plugs In	8
5. Canon Power-Level System — The Data Backbone	9
5.1 The Data Model	9
5.2 The Combat-Ratio Damage Formula	9
5.3 Why Linear PL Fails (and Noea Avoids It)	9
5.4 Verified Canon Reference Table (Saga Peaks)	9
6. RPG Progression Layer — The Noea Inheritance	11
6.1 Races and Form Ladders	11
6.2 The Six-Stat Sheet → Battle Power	11
6.3 Training Points — The Universal Currency	11
6.4 Power Release — The In-Combat Throttle	12
6.5 Mastery — The Form-Use Curve	12
7. Transformation System — Merging Two Models	13
7.1 The Two Models Side by Side	13
7.2 Why This Merge Works	13
7.3 The Canon-Accurate Form Multipliers	13
8. Architecture Blueprint — Engine and Core Systems	14
8.1 Engine Choice — Unreal Engine 5	14
8.2 Core System Decomposition	14

8.3 Data-Driven Character Definition	14
9. Combat System Implementation — Frame Data and State Machines	16
9.1 Animation State Machine Design	16
9.2 Frame Data — The Move Definition Schema	16
9.3 Hitboxes and Hurtboxes	16
9.4 The Counter Implementation	16
10. Netcode — Rollback for PvP, Relay for Co-op	18
10.1 Why Rollback, Not Delay	18
10.2 The Determinism Requirement	18
10.3 Co-op and Spectator	18
11. Destructible Terrain & Planet Destruction	19
11.1 Voxel Terrain with GPU Marching Cubes	19
11.2 The Planet-Integrity Model	19
11.3 Star Destruction Cinematic	19
12. Content & Saga Structure	20
12.1 The Saga Backbone	20
12.2 Masters and Training Minigames	20
12.3 Tournament Bracket Mode	20
12.4 Custom Raid Bosses	20
13. Build Roadmap — Phased Plan	21
13.1 Phase 1 — Combat Prototype (Weeks 1-8)	21
13.2 Phase 2 — Vertical Slice (Weeks 9-20)	21
13.3 Phase 3 — Beta (Weeks 21-44)	21
13.4 Phase 4 — v1.0 (Weeks 45-60)	21
13.5 Effort and Team-Size Estimate	21
14. Risks, Legal, and Caveats	23
14.1 Trademark and Legal	23
14.2 The "Original IP" Alternative	23
14.3 Scope-Creep Warnings	23
14.4 The "Sparking Clone vs Original" Line	23
15. Bibliography — Research Sources	24
15.1 Primary Source PDFs (supplied / built in this workspace)	24
15.2 Canon Power-Level Sources (cited in the fact-checked guide)	24
15.3 Game-Design and Engineering Sources (web research)	24

1. Executive Summary — The Game Concept

This document specifies how to build a Dragon Ball game that combines the three things the research established as the genre's best practices: the **combat feel** of *Dragon Ball: Sparking! ZERO* (Bandai Namco, 2024), the **progression depth** of the fan-made *Dragon Block Noea / DragonMineZ* Minecraft mod ecosystem (Forge 1.20.1, 2024-2026), and the **canon-accurate power levels** documented in the fact-checked power-scaling guide (sourced to the manga, Daizenshū 7, and V-Jump).

The core thesis: **Sparking! ZERO's combat is effective because it is not a combo-memory game — it is a positioning and resource-read game.** The 3D arena, the Ki economy, and the layered counter system make every exchange a series of small reads rather than one long executable string. Layering Noea-style RPG progression (races, six-stat sheet, Training Points, Power Release, mastery) on top gives the player something to *do between fights*, and canon power levels give every number a defensible meaning instead of an arbitrary one.

1.1 The Three Pillars

Pillar	Source game/mod	What it contributes	Research basis
Combat	Sparking! ZERO	3D arena movement, Ki economy, layered counter system, Sparking! ZERO Combat Guide, Transformation	Sparking! ZERO Combat Guide, Transformation
Progression	Dragon Block Noea / DragonMineZ	Races, Six-stat sheet, Training Points, Power Release, mastery	DragonMineZ Research Guide (8 pp), space/plane
Data	Canon power-scaling	Every character's PL tagged to manga/Daizenshū 7, and V-Jump	Fact-Checked Power Levels (12 pp), formula; c

Each pillar is independently proven. *Sparking! ZERO* shipped to strong reviews for its combat depth; *DragonMineZ* has 538,000+ CurseForge downloads; the canon power-level guide cross-references every value against *Kanzenshū*, the manga's own scouter readings, and *Daizenshū 7*. Combining them is the design challenge this document solves.

1.2 The One-Sentence Pitch

"A 3D arena Dragon Ball fighter where every number on the screen means something real — your power level is canon-accurate, your training earned it, and every fight is a read-and-punish duel in the sky, not a combo-execution test."

2. The Three Source Systems — What Each Teaches

Before specifying the merged design, it is worth establishing exactly what each source system does well and where each is weak. The merged game takes the strong half of each and discards the weak half.

2.1 Comparative Matrix

Dimension	Sparking! ZERO	Dragon Block Noea	Canon PL research
Genre	3D arena fighter (fighting game)	Character-progression RPG (Reference)	Reference dataset
Combat depth	Very high — layered counter-low Ki-auto-attack + ability	Low Ki-auto-attack + ability	None (data)
Progression	Flat — 5-point stat spread	Deep character stats + TP + mastery	None
Power accuracy	Loosely canon-flavored (stated on-panel)	Canon faithful multipliers (x50/x100/x400) ceiling extends past canon	None
Content scope	164-character roster, story	Solo offhanded sagas, space, planets	Manga, chads 325 + DBS
Multiplayer	Online PvP (delay/rollback)	Minecraft server co-op	N/A
Built with	Unreal Engine 5 (Bandai)	Minecraft Forge + GeckoLib	N/A
Strength to take	Combat feel, counter layer	RPG systems, transformation	Damage math, PL data
Weakness to drop	Shallow progression, no trading	Shallow combat, auto-attack	No game — pure reference

2.2 What "Accurate Power Levels" Means in Practice

The fact-checked canon guide establishes a five-tier source hierarchy: [C] manga-stated on-panel, [D] Daizenshuu 7 (Shueisha's official databook), [V] V-Jump magazine promo values, [G] game tie-in (Kakarot / Scouter Battle Taikan Kamehameha), [S] community scaling via canon multipliers, and [!] contested where sources disagree. A faithful game should treat these tiers as data flags on every character record — so a modder or designer can see at a glance whether a number is Toriyama-stated or community-derived, and tune accordingly.

Tier	Source	Example value	Reliability
[C]	Manga (on-panel scouter)	Raditz 1,500 — Vegeta 18,000 — Frieza 1st 530,000	Highest
[D]	Daizenshuu 7	Goku SSJ 150,000,000 — Frieza 100% 120,000,000	High (official databook)
[V]	V-Jump magazine	SSJ3 Goku 24,000,000,000	Medium (promo, sometimes inconsistent)
[G]	Game tie-in	Android 17/18/16 V-Jump-style values	Low (often inconsistent)
[S]	Scaling via canon multipliers	SSJ2 Gohan ~1.8B (x100 of ~7M base, manga-stated)	Medium-higher
[!]	Contested	Nappa 4,000 (Daizenshuu vs manga showings); Frieza 1st 530,000 (Daizenshuu vs 24B (manga evidence))	Flagged

Takeaway: tag every power-level field in the character database with its source tier. This makes tuning, modding, and balance review auditable — and lets players who care about canon verify any number.

3. Why Sparking! ZERO's Combat Is So Effective

Independent reviews (CNET, Rolling Stone, Hardcore Gamer) and the player-base consensus on GameFAQs / Steam all converge on the same read: Sparking! ZERO is "anime fighting perfection" specifically because its combat is **simple to learn and takes time to master**, and because it produces a **theatrical look** (the source of the Steam-forum complaint that the system "neuters itself once you learn it" — which is the same property, viewed from the casual side). The design rests on five interlocking choices.

3.1 Reason 1 — Movement is the backbone, not combos

Sparking! ZERO is described by its own guide as "*closer to footsies in the sky than a traditional 2D fighter*". The kit is built around **Step/Short Dash** (spacing), **Dragon Dash** (close distance, Ki-cost), **Z-Burst Dash** (reposition behind the opponent for extra Ki — the engine behind the hard-to-block Burst Rush), and **Quick Ascend/Descend** (vertical fighting, which several attacks specifically exploit). The result: **positional reads matter more than memorized strings**. A player who never learns a single long combo can still win by out-positioning and out-resource-managing the opponent. This is the single biggest lesson to take from Sparking! ZERO — the combat ceiling is set by movement, not by combo length.

Design implication: the character controller must make movement feel good before any attack is wired up. If the dash, step, fly and ascend don't feel responsive, the entire combat model collapses. This is the first system to prototype and the last to polish.

3.2 Reason 2 — Ki is the universal economy

Ki is the single resource that gates *everything*: Ki Blasts, Blast Attacks (Super Attacks), Dragon Dash, Z-Burst Dash, Quick Ascend/Descend, Vanishing Assault follow-ups, Super Perception (sometimes), and the path to filling the Sparking Gauge. There is no separate "stamina" or "meter" pool fragmenting attention — just Ki. The GameFAQs thread puts it directly: "*Ki management is the name of the game here. If you win the war, you get smash damage and oki advantage.*"

The reason this works: **one resource = one decision tree**. Every action can be evaluated against every other action on the same scale ("does this Ki spend beat that Ki spend?"), which makes the game readable for spectators and learnable for newcomers. Splitting resources (stamina + super meter + burst gauge + transformation stock, as some fighters do) fragments the read and raises the floor without raising the ceiling.

3.3 Reason 3 — Layered counters with different costs

The defense tree is where the skill ceiling actually lives. Sparking! ZERO layers five counter tools, each with a different cost and a different best-case target — so defense is never "guard or die":

Counter	Cost	Best against	Risk
Super Perception	Ki, or 2 Skill Count vs. Blast Attacks (Sonic Sway on perfect timing)	Any character	Free cancelable
Z-Counter	Timing only	Vanishing Assault — escalating damage during	Mistime = take the hit
Revenge Counter	1 Skill Count	Being actively combo'd, any direction	Predictable — baitable by free Perception
Super Counter	Precise timing, no resource cost	Almost any attack, incl. from behind / mid-air	Hardest timing in the game
High-Speed Evasion / Step-Twisting	Timing only	General attack avoidance	No punish on whiff

The intended defensive loop, per the guide: *guard or evade against unclear pressure* → *use Perception or Z-Counter to punish predictable offense for free* → *save Revenge/Super Counter as a Skill-Count-backed panic button once you're actually being combo'd*. This is a **tiered risk ladder** — the player always has a cheaper option that requires more skill, and a more expensive option that requires less. That ladder is what produces the "simple to learn, hard to master" curve.

3.4 Reason 4 — Sparking! Mode is the shared climax

Sparking! Mode (Sparking Gauge full + ≥ 1 Skill Count) is the game's comeback / payoff state: it unlocks the Ultimate Blast, makes normally Ki-costing actions free, grants Ki-Blast-stun immunity, and adds damage/speed/HP-regen. The advanced sequence — **Burst Rush** → **Burst Finish** → **Sparking Combo** → **Ultimate Blast** — is the canonical "win condition" both players are racing toward.

Why this is effective: it gives every match a **shared dramatic arc**. Both players can see the Sparking Gauge fill; both know the climax is coming; the question is who gets there first and whether they can land the setup. This is the structural reason Sparking! ZERO feels "theatrical" — the resource economy is literally a three-act structure (neutral → Sparking race → Ultimate Blast resolution).

3.5 Reason 5 — Every character is a "setup archetype"

A Steam-community observation that captures the design intent: *"Every character is a setup archetype. It is designed to work off these systems to make a theatrical look."* Unlike traditional fighters where characters are differentiated by their combo strings (rushdown / zoner / grappler), Sparking! ZERO differentiates by **what they want to set up** — which Blast Attack, which Ultimate, which transformation chain. The shared core kit means a player who learns the system can pilot any character; the character select is a question of *preference in setup style*, not of learning a new control scheme.

Takeaway for our build: keep the core combat kit identical across all characters. Differentiate by (a) Blast/Ultimate moveset, (b) transformation ladder, (c) stat-spread weights, (d) unique passives. Do NOT give different characters different control schemes — that fragments the read for both players and spectators.

4. The Combat Loop — Anatomy of One Exchange

A high-level Sparking! ZERO exchange, as documented in the official Bandai beginner guide and breakdowns by TheGamer / NoobFeed / GamingBolt, follows a repeatable five-phase loop. Mapping it explicitly is the foundation of the merged game's combat design — every system in the build must serve one of these five phases.

4.1 The Five-Phase Loop

Phase	Player goal	Primary tools	Resource state
1. Neutral	Bait a guard or force movement with Ki Blasts, Spinning Attacks at range	Ki Blasts, Spinning Attacks	Ki building up
2. Approach	Close distance; punish a whiffed/over-guarded move	Dragon Dash, Z-Burst Dash	Ki spent on dash
3. Conversion	Convert one read into a full offensive	Versus Charge Assault → Dragon Homing / Versus Charge Assault	Skill Count / Lightning Attack
4. Defense read	Defender picks the right counter from the read	Super Guard / Z-Counter / Revenge Super Guard	Ki Super Guard if panic
5. Climax	Race to Sparking Mode; land Ultimate Blast	Super Guard Mode → Burst Rush → Burst Finish	Sparking Gauge / Skill Count

The loop is **symmetric** — both players are running it simultaneously, which is what makes the game a duel rather than a single-player combo exhibition. The Ki economy (Phase 1), the Skill Count bank (Phase 3), and the Sparking Gauge (Phase 5) are three resource clocks ticking in parallel, and the player who reads their opponent's clock-state best wins.

4.2 Where the RPG Layer Plugs In

The canon power-level system and the Noea-style RPG layer sit at **two specific points** in this loop, not across all of it — which is why they can be added without breaking the combat feel:

Plug-in point	What the RPG/canon layer adds	Effect on combat feel
Pre-match (loadout)	Race + six-stat sheet + trained PL + chosen transformations	Generates the damage ratio and form options — but the in-game damage numbers are the same
Phase 3 (Conversion)	Mid-battle transformation costs Skill Count + shifts strategy (Sparking decision)	Costs Ki in preparation
Phase 5 (Climax)	Ultimate Blast is the form-appropriate finisher (Kaio-ken, Final Flash, Big Bang Attack)	The payoff for the Final Flash / Big Bang Attack

The combat loop itself is untouched by the RPG layer. A Goku at 416 PL and a Goku at 150,000,000 PL pilot the same character controller with the same moves — only the damage numbers and the available forms differ. This is exactly how Sparking! ZERO handles its 164-character roster, and it is the only sane way to merge an RPG grind with a fighting game.

5. Canon Power-Level System — The Data Backbone

This chapter specifies how to turn the fact-checked canon power levels into a working in-game damage system. The core problem: canon PL spans **seven orders of magnitude** (Farmer 5 → Vegito ~70 billion), and a linear damage formula makes 99% of matchups one-shot kills. The solution is a **log-scaled combat ratio**, which is exactly the approach the Dragon Block Noea mod takes ("damage scales with effective PL vs enemy PL") and which the dragon-power.js prototype in this very workspace implements.

5.1 The Data Model

Every character record carries the following fields, each tagged with the source tier from Chapter 2.2:

```
CharacterRecord { id: "goku" name: "Goku" race: SAIYAN basePL: 416 // [C] manga ch.215 (weighted)
basePLSource: "MANGA_CH215" forms: [ { name:"Base", plMul:1, tier:"[C]" }, { name:"Kaio-ken",
plMul:4, tier:"[C]" }, // manga-stated { name:"Super Saiyan",plMul:50, tier:"[D]" }, // Daizenshuu 7
{ name:"SSJ2", plMul:100, tier:"[D]" }, { name:"SSJ3", plMul:400, tier:"[D]" }, ] statSpread: {
STR:7, SKP:8, RES:7, VIT:8, PWR:9, ENE:7 } // /10 passives: ["ZENKAI"] blast: "Kamehameha" ultimate:
"Super Kamehameha" }
```

The **effectivePL** at any moment is $\text{basePL} \times \text{form.plMul} \times \text{powerRelease\%}$, exactly mirroring the Noea mod's Power Release throttle (raw stats are not fully "on" by default — release charges from 0% to a 50% cap, raised to 100% by the Potential Unlock passive).

5.2 The Combat-Ratio Damage Formula

The damage formula must compress seven orders of magnitude into a playable range. The proven approach (used by Noea, used by the dragon-power.js prototype in this workspace, and recommended by RPG-design literature for "ratio-based" scaling) is:

```
ratio = attacker.effectivePL / max(defender.effectivePL, 1) ratioScale = log10(ratio + 9) // 1x →
~1, 10x → ~2, 100x → ~3, 1000x → ~4 baseDamage = move.baseDamage * ratioScale * move.typeMultiplier
finalDamage = baseDamage * (1 - defender.guardReduction) // example: Goku 416 (Kaio-ken x4 = 1664)
vs Vegeta 18000 // ratio = 1664/18000 = 0.092 // ratioScale = log10(0.092 + 9) = log10(9.092) ≈
0.959 // → Goku hits Vegeta for ~baseDamage × 0.96 (he's outmatched ~11x, hits ~4% weak) // example:
SSJ Goku 150,000,000 vs Frieza 100% 120,000,000 // ratio = 1.25, ratioScale = log10(10.25) ≈ 1.01 //
→ roughly even exchange, slight Goku edge
```

The log curve means: a 10× PL advantage is a meaningful but not fight-ending edge (~1.0× base damage vs ~0.95× for an even match); a 100× advantage is decisive but the underdog still does *some* damage; a 1000× advantage is a one-shot. This **preserves the drama of the source material** — Raditz torturing Goku and Piccolo (1,500 vs 416/408, ~3.6× advantage) plays out as a real fight, not a one-hit kill, which is exactly what the manga shows.

5.3 Why Linear PL Fails (and Noea Avoids It)

A naive $\text{damage} = \text{attackerPL} - \text{defenderPL}$ or $\text{damage} = \text{attackerPL} / \text{defenderPL} \times \text{base}$ formula produces absurd results: Goku at 416 vs Frieza at 120,000,000 would do 0.00035% of a hit, and the reverse would be a 288,000× one-shot. The log-scaled ratio is the minimum viable compression. Noea's own implementation (per the Noea guide: "damage scales with effectivePL vs enemyPL") uses the same family of formula, which is why its 200M vs 600M immortal-vs-Vegeta demo still produces a watchable exchange rather than an instant kill.

5.4 Verified Canon Reference Table (Saga Peaks)

The following peaks are the canon-verified damage anchors for the saga-difficulty curve. Every other value in the character database scales multiplicatively from these via the Daizenshuu 7 multipliers (×50 SSJ, ×100 SSJ2, ×400 SSJ3) and the manga-stated form relationships.

Character (peak form)	PL	Source	Saga
-----------------------	----	--------	------

Raditz	1,500	[C] Daizenshoo 7	Saiyan
Vegeta (Saiyan saga)	18,000	[C] manga ch.249	Saiyan
Ginyu	120,000	[C] manga ch.285	Namek
Frieza 1st form	530,000	[C] manga ch.294	Namek
Frieza 100%	120,000,000	[D] Daizenshoo 7	Namek
Goku SSJ	150,000,000	[D] Daizenshoo 7	Namek
SSJ2 Gohan	~1,800,000,000	[S] (> Super Perfect Cell ~1.6B)	Cell
SSJ3 Goku	24,000,000,000	[V] V-Jump	Buu
SSJ3 Gotenks	~30,000,000,000	[S] (> SSJ3 Goku, manga-stated)	Buu
Ultimate Gohan	~35,000,000,000	[S] (> Super Buu, > Gotenks, > Goku)	Buu
Kid Buu	~24,000,000,000	[V]/[S] ($\approx$ SSJ3 Goku, manga equality)	Buu
Vegito (base)	~70,000,000,000+	[S] (> Buuhan)	Buu

Contested values (Nappa 4,000; the V-Jump Android 17/18/16 numbers; Kid Buu 1.15B from the arcade game) must be flagged [!] in the database and presented with both numbers, preferring the manga equality (Kid Buu $\approx$ SSJ3 Goku $\approx$ 24B) over the arcade value. See the fact-checked guide for the full corrections log.

6. RPG Progression Layer — The Noea Inheritance

The progression system is adapted directly from DragonMineZ (the base mod) and Dragon Block Noea (the add-on), because that ecosystem has already solved the hard problem of making a Dragon Ball RPG feel like Dragon Ball — race choice that matters, a six-stat sheet that feeds a single Battle Power number, Training Points as the universal progression currency, Power Release as the in-combat throttle, and a mastery curve that rewards using a form repeatedly.

6.1 Races and Form Ladders

Race choice fixes two things: the **passive skill** and the **core transformation ladder**. Visual customization (hair, eyes, aura color) is cosmetic and does not affect stats — exactly as in DragonMineZ.

Race	Passive	Core transformation ladder	Notes
Human	Recharge (faster Ki charge)	Bufed → Full Power → Potential Unleashed → Ears on the Loose → Kaibutsu → Ultra Instinct	Can be unlocked via Kaibutsu, Ultra Instinct; Gamr
Saiyan	Zenkai (small permanent boost to all stats)	Stoza part 1 → Seal Grade 2 → Death Master → SSB → SSJ → SSJ Rosé	SSB → SSJ → SSJ Rosé gated behind
Namekian	Meditation (faster passive Ki regen)	Bufed → Full Power → Potential Unleashed → God of Destruction	God of Destruction chain via God Ki
Bio-Android	Absorption (heals % of damage taken)	Cell → Perfect Cell → Super-Perfect → Ultra	Temporary Cell Max via Shenron wish; can l
Cold Demon (Frieza)	Muscle (higher TP gain, boosted Ki)	Full Power → 5th Form	Golden (Final/5th); Black Frieza (manga DB)
Majin	Regeneration (passive HP regen)	Bufed → Kid → Super → Ultra	Reworked absorption ("margin" mechanic)

6.2 The Six-Stat Sheet → Battle Power

DragonMineZ uses six core attributes, opened with the stats menu. The three offensive stats stack additively into total offensive output; the three defensive stats build survivability and resource pools. All of it feeds a single derived number — **Battle Power**, the modern equivalent of a classic Dragon Ball power level.

Stat	Drives	Stacks into
STR (Strength)	Melee (physical) damage	Offense
SKP (Strike Power)	Ki-enhanced melee / technique "strike" damage	Offense
PWR (Ki Power)	Ki blast / energy-wave damage	Offense
RES (Resistance)	Defense/damage mitigation + max Stamina + max Poise	Survivability
VIT (Vitality)	Max Health + Health/Stamina regen rate	Survivability
ENE (Energy)	Max Ki (energy pool)	Survivability

Battle Power = f(offense_stats) × powerRelease% × form.pIMul × masteryBonus. This is the number that climbs into the billions (Noea's narrator cites 135 billion at the god-tier ceiling) and that feeds the combat-ratio formula from Chapter 5.2.

6.3 Training Points — The Universal Currency

TP is spent on stats, skills, Ki techniques, and transformation mastery. It is earned from parallel sources, all run through TP-boost multipliers:

- **Kills** — TP scales with the target's Battle Power, so fighting stronger enemies is far more efficient than farming weak mobs (anti-grind design).
- **Landed hits** — small TP per combat hit, independent of the kill (rewards aggressive play).
- **Passive trickle** — small constant gain just for playing (rewards time-in-game).
- **Travel / block-breaking / crafting** — each awards TP, scaled to distance/amount (rewards exploration).

- **Training minigames** — five NPC Masters each teach one (Rhythm/Mr. Popo, Control/Krillin, Shadow Boxing/Gohan, Precision/Trunks, Gravity/Vegeta); rewards scale with current power up to a per-session cap — the single biggest deliberate TP source.

- **Environment multipliers** — Gravity Devices, the Hyperbolic Time Chamber, and weighted training gear all multiply TP gain; Cold Demons get a passive $\times 1.25$ TP bonus.

6.4 Power Release — The In-Combat Throttle

A character's raw stats are not fully "on" by default. Holding the release key charges Power Release from 0% up to a base cap of 50%; the Potential Unlock passive (taught by King Kai) raises that ceiling to 100%. Release is a **throttle on how much of the underlying stat sheet is expressed in combat at any given moment** — low release means weaker attacks and lower visible Battle Power even though the trained stats haven't changed. This is the mechanical bridge between the RPG grind and the fighting-game match: a player who has trained hard but doesn't charge release enters the fight at half-strength.

Design implication for the merged game: Power Release is the player's "I'm taking this fight seriously" button. It maps cleanly onto the Sparking! ZERO Ki-Charge mechanic — hold to fill, vulnerable while charging, fills the resource that gates your peak output.

6.5 Mastery — The Form-Use Curve

Each transformation has a TP unlock cost and, while active, an ongoing **Ki-drain upkeep**. Repeated use of a form builds Mastery, which both *reduces* that upkeep and *increases* the form's stat multiplier — a fully mastered form can reach up to $\times 1.5$ stats while upkeep drops to $\times 0.75$ of its base cost, and some forms even become free to hold once fully mastered. Related forms can share mastery progress.

This is the single best idea in the Noea system and must be preserved wholesale. It does three jobs at once: (1) it gives the player a long-term reason to use forms they've "outgrown" stat-wise; (2) it produces a satisfying mastery curve that mirrors the anime (Goku's first SSJ3 is exhausting; his later ones are stable); (3) it gives the form a second balance dial (upkeep + multiplier) instead of just one, making balance tuning far more expressive.

7. Transformation System — Merging Two Models

Both source systems handle mid-battle transformation, but differently. The merged game takes **Sparkling! ZERO's resource model** (transformation costs Skill Count, shifts the 5-point stat spread) for the in-fight layer, and **Noea's mastery model** (Ki-drain upkeep, mastery curve) for the progression layer. The two coexist because they operate at different time scales.

7.1 The Two Models Side by Side

Property	Sparkling! ZERO model	Noea model	Merged game uses
Cost to activate	Skill Count stock	TP unlock (one-time) + Ki-drain upkeep (ongoing)	Skill Count (fight) + TP unlock (meta)
Effect	Shifts the 5-point stat spread	Staty upkeep + Ki-drain	Stat spread shift + multiplier + upkeep
In-fight limitation	None — form persists until reverted	Ki-drain upkeep; form drops when Ki hits zero	Ki-drain upkeep (Noea-style) — form drops at 0
Long-term progression	None — flat per character	Mastery curve reduces upkeep	Mastery multiplier (Noea-style)
Climax interaction	Transformed → Sparking Mode	Ultimate (Kamehameha)	Appropriate Ultimate Blast (canon-accurate)

7.2 Why This Merge Works

Sparkling! ZERO's "transformation = Skill Count spend" is a clean **in-fight** resource decision — do I transform now, or save the Skill Count for a Revenge Counter? But it has no long-term arc: a Goku who has used SSJ3 a thousand times pilots it identically to one using it for the first time. Noea's mastery model is a clean **meta** arc — your SSJ3 gets better the more you use it — but its in-fight transformation is just "press key, pay Ki."

The merge: **in-fight, transformation costs Skill Count and immediately shifts your stat spread** (Sparkling) **AND imposes a Ki-drain upkeep that drops you out of form if Ki hits zero** (Noea). **Meta-game, the mastery you've earned on that form reduces the upkeep and raises the multiplier** (Noea). This gives the player both a per-fight resource puzzle and a long-term training arc, and neither breaks the combat loop from Chapter 4.

7.3 The Canon-Accurate Form Multipliers

Every form multiplier in the database must trace to its canon source. The Daizenshoo 7 + Super Exciting Guide chain is the backbone:

Form	Multiplier (over base)	Source	Notes
Base	×1	—	—
Kaio-ken	×4 (×2 base, ×2 Kaio-ken)	[C] manga	Goku vs Vegeta; scales ×2/×3/×4/×10/×20
Super Saiyan	×50	[D] Daizenshoo 7 p.44	SSJ Goku 150M = 3M base × 50
SSJ2	×100	[D] Daizenshoo 7 + SEG	×2 over SSJ
SSJ3	×400	[D] Daizenshoo 7 p.83	×4 over SSJ3 (= ×8 over SSJ)? contested; SEG says
Oozaru	×10	[C] manga	Vegeta 18,000 → ~180,000 Oozaru
Golden Oozaru	×500 (×50 Oozaru)	[S] scaling	SSJ4 precursor (GT scaling)
Super Saiyan God	×? (god-tier)	[G] game tie-in	Below SSB; exact multiplier not canon-stated
Ultra Instinct (Sign)	~10% dodge → ~70% at mastery	[S] add-on ecosystem	Mastery-scaled dodge chance
Ultra Instinct (Mastered)	~40% → ~80% at mastery	[S] add-on ecosystem	Top of the dodge ladder
Ultra Ego	rage/damage-scaled	[S] add-on ecosystem	Vegeta's god form

God-tier multipliers (SSG/SSB/UI/UE/Beast/Rosé) are NOT canon-stated as hard numbers — the manga and Daizenshoo stop at SSJ3 ×400. Any number above that is game-balanced or community-scaled and must be tagged [S] or [G], never presented as canon.

8. Architecture Blueprint — Engine and Core Systems

This chapter specifies the engineering architecture. The research (Unity vs Unreal forum consensus, UE5 fighting-game tutorial ecosystem, the fact that Bandai Namco builds Sparking! ZERO on Unreal Engine 5) points to a clear engine choice and a clean system decomposition.

8.1 Engine Choice — Unreal Engine 5

For a Sparking! ZERO-style 3D arena fighter, **Unreal Engine 5 is the correct choice**. The reasons:

- **AAA graphics out of the box** — Nanite + Lumen produce the cinematic anime-cel-shaded look Sparking! ZERO ships with; Unity can match it only with significant custom shader work.
- **Built-in fighting-game references** — the UE5 "Fighting Game" tutorial series and the Universal Fighting Engine (UFE) for Unity both exist, but UE5's Chaos physics, Niagara VFX, and Animation Blueprint system are better suited to the destructible-terrain + transformation + aura-VFX load this game demands.
- **Bandai uses it** — Sparking! ZERO itself is UE5, meaning the reference target runs on the same engine; shaders, animation pipelines, and netcode patterns can be directly informed by what Bandai shipped.
- **Gameplay Abilities System (GAS)** — UE5's GAS is a data-driven ability framework that maps almost 1:1 onto the Noea skill tree (skills leveled with TP, with tags, costs, and cooldowns). It is the right tool for the RPG layer.

Caveat: if team size is small and C++ is a blocker, Unity (with the Universal Fighting Engine asset and a custom RPG layer) is the viable second choice. The architecture below is engine-agnostic in shape; only the implementation library names change.

8.2 Core System Decomposition

The game decomposes into nine core systems. Each is independently testable and independently replaceable — a modder (or a future developer) can swap any one without touching the others.

#	System	Responsibility	Key tech
1	Character Controller	Movement (walk/dash/fly/ascend), camera, input mapping	UE5 CharacterMovementComponent
2	Combat System	Move execution, hitboxes, damage, guard, counter	Custom + GAS for abilities
3	Power-Level Engine	effectivePL calc, combat-ratio damage formula, soul meter	Custom + GAS for abilities
4	Progression System	Races, six-stat sheet, TP, Power Release, mastery	GAS + GameplayTags + SaveGame
5	Transformation System	Form state, stat-spread shift, Ki-upkeep, mastery	Custom system of Combat + Progression
6	Animation System	State machine, blend trees, frame data, cancels	UE5 Animation Blueprint + AnimMontage
7	Netcode	Rollback for PvP; relay for co-op; spectator	GGPO-style rollback layer
8	World Systems	Space travel, planets, destructible terrain, raids	Voxel/marching-cubes + Niagara
9	Content layer	Saga story, masters, tournaments, dialogue	Data tables + quest system

8.3 Data-Driven Character Definition

Characters, moves, and forms are **data, not code**. A designer (or modder) adds Goku by editing a data table, not by writing a class. This is how Sparking! ZERO ships 164 characters and how Noea ships its full form ladders — and it is the only way the canon power-level data stays maintainable.

```
// DataTable: /Game/DB/Characters/Goku.uasset BasePL=416 BasePLSource="MANGA_CH215" Race=SAIYAN
Forms=[ Base, Kaio-ken(x4), SSJ(x50), SSJ2(x100), SSJ3(x400) ]
StatSpread={STR:7,SKP:8,RES:7,VIT:8,PWR:9,ENE:7} Passives=[ZENKAI] Blast=Kamehameha
Ultimate=SuperKamehameha // DataTable: /Game/DB/Moves/Kamehameha.uasset Type=BLAST BaseDamage=42
```


9. Combat System Implementation — Frame Data and State Machines

The combat system is the heart of the game and the part most likely to be under-engineered. The research on fighting-game state machines (gamedev.net, mocaponline.com on 200+ state systems, the Universal Fighting Engine framework) converges on a small set of principles.

9.1 Animation State Machine Design

The single most important principle from the research: **"use as few states as possible"** (gamedev.net). A naive fighting game has 200+ animation states (every move × every cancel × every follow-up); the correct architecture layers rather than enumerates.

Layer	What it holds	Tech
Locomotion	Idle / walk / run / fly / dash — continuously blended by speed & direction	Blend trees (2D/3D)
Action	Punch / ki-blast / smash / throw — discrete montages with finite edges	Animation montage + state
Reaction	Hit-stun / guard / knockback / getup — driven by the combat system transitions	State, system, player
Transformation	Form-change cinematic + stat-shift — overlays on top of action layer	Overlay layer
Aura/VFX	Sparking! glow, Ki aura, charge particles — cosmetic, a Niagara+socket layer	Niagara+socket

Blend trees handle continuous locomotion; a discrete state machine handles discrete actions. The two do not need to know about each other. mocaponline's "patterns for 200+ states" research is explicit that layering (not a single flat FSM) is the only sustainable architecture.

9.2 Frame Data — The Move Definition Schema

Every move is defined by frame data, exactly as in traditional 2D fighters (and as UFE implements). The schema:

Phase	Frames	What happens
Startup	N frames	Move is animating but no hitbox active — interruptible here is a "gimp"
Active	M frames	Hitbox live; connects if it overlaps a hurtbox
Recovery	R frames	Hitbox gone; cannot act — the "whiff punish" window
Cancel windows	specific frames	Frames where the move can be cancelled into a designated follow-up
On hit / on block	+/- frame advantage	frame advantage the attacker has after the move resolves

At 60 FPS, Sparking! ZERO's timing windows translate roughly to: Smash Attack charge $\approx$ 30 frames (0.5s), Super Counter window $\approx$ 4–6 frames (the "hardest timing in the game" per the guide), Z-Counter window $\approx$ 8–10 frames. These windows are the skill-ceiling knobs — tightening them raises the ceiling, loosening them lowers it.

9.3 Hitboxes and Hurtboxes

Every animated move carries one or more **hitboxes** (the damage volume) and every character carries **hurtboxes** (the damageable volumes). The collision is sphere/capsule-based for performance (UE5 supports thousands of these per frame). The research consensus: **do not use mesh-accurate hitboxes** — they are visually impressive but make the game unreadable (a fist that whiffs by 2mm feels like a bug). Use generous capsules that match the visual intent, not the visual geometry.

9.4 The Counter Implementation

The five-counter ladder from Chapter 3.3 is implemented as a state-check on the defender's input during the attacker's Active frames:

```
onDefenderInput(frame, incomingMove): if frame in SuperCounterWindow: # ±3 frames, no cost return  
COUNTER_SUPER if frame in ZCounterWindow and incomingMove.type == VANISHING_ASSAULT: return  
COUNTER_Z # escalating damage duel if frame in RevengeWindow and defender.skillCount >= 1: return  
COUNTER_REVENGE # costs 1 Skill Count if frame in PerceptionWindow: if incomingMove.type == RUSH:  
return PERCEPTION_SONIC_SWAY # free elif incomingMove.type == BLAST and defender.skillCount >= 2:  
return PERCEPTION_BLAST_COUNTER # costs 2 Skill Count if frame in EvasionWindow: return EVASION #  
timing only return GUARD_OR_HIT
```

10. Netcode — Rollback for PvP, Relay for Co-op

Netcode is the single most consequential engineering decision after engine choice. The research (Ars Technica's rollback explainer, infil.net's fighting-game glossary, the rollback-vs-delay debate on ResetEra / teamfortress.tv) is unambiguous: **PvP fighting games must use rollback netcode; delay-based netcode feels bad in practice and is the wrong choice for a read-heavy game.**

10.1 Why Rollback, Not Delay

In **delay-based** netcode, the local game waits for the opponent's input to arrive before advancing the simulation. This adds input latency proportional to ping — a 100ms connection makes a 4-frame Super Counter window literally unreactable. In **rollback** netcode, the local game never waits: it advances the simulation using the local player's real input and the opponent's *predicted* input. When the opponent's real input arrives, if it differed from the prediction, the game rolls the state back to the divergence point and re-simulates forward with the correct inputs. The result (per Ars Technica): *"rollback's main strength is that it never waits for missing input from the opponent. Instead, rollback netcode continues to run the game."*

10.2 The Determinism Requirement

Rollback requires that the game simulation is **fully deterministic** — given the same inputs and the same starting state, the simulation must produce the identical ending state, bit-for-bit, on both machines. This is the hard part. It means:

- **No floating-point non-determinism** — UE5 floats are not deterministic across CPUs; the combat simulation must use fixed-point math (e.g. 16.16 fixed) or a deterministic float library.
- **No unlocked-thread race conditions** — the simulation runs on a single thread; rendering and VFX can be on other threads, but the gameplay state must be single-threaded.
- **No system-time randomness in combat** — all RNG must come from a shared seeded PRNG synchronized across both clients.
- **State save/restore** — the game must be able to snapshot its full state every frame and restore it in <2ms for the rollback. This is what makes the architecture demanding.

10.3 Co-op and Spectator

For co-op modes (saga co-op, raids — the Noea-style content), strict determinism is overkill: a small state desync is invisible when players aren't reading each other frame-by-frame. Use a **relay server with periodic state reconciliation** for co-op, reserving rollback for the 1v1 PvP ladder. Spectator mode is a third path: the server (or a designated client) runs the authoritative simulation and broadcasts a compressed state stream to spectators at 20Hz, with client-side interpolation.

11. Destructible Terrain & Planet Destruction

The Noea mod's headline feature — and the one that most distinguishes a Dragon Ball game from a generic fighter — is destructible terrain and planet destruction. The research on voxel/marching-cubes terrain (Unity discussions, gamedev.net, benwindley.github.io, GitHub "Marching-Cubes-On-The-GPU") establishes the implementation path.

11.1 Voxel Terrain with GPU Marching Cubes

Destructible terrain requires a **voxel representation** (a 3D grid of density values) rendered to a mesh via the **marching-cubes algorithm**. The canonical approach, per the GitHub project and benwindley's write-up: run marching-cubes in a **compute shader** on the GPU, generating the mesh in a vertex buffer that updates only when voxels change. A Kamehamha carve becomes: write zeros to a voxel region → re-run marching cubes for affected chunks → the mesh updates.

Property	Value	Note
Voxel resolution	0.5m – 2m cubes	Fine enough for ki-blast carves; coarse enough to render
Chunk size	16 ³ or 32 ³ voxels	One GPU dispatch per chunk per rebuild
Rebuild scope	Only chunks that changed	Differential update — not the whole planet
Storage	3D texture / StructuredBuffer	GPU-resident; CPU only for saves
Planet size	500–5000 block radius (Noea spec)	Seamless spherical wrap (Noea v1.0)

11.2 The Planet-Integrity Model

Noea's planet-destruction model (per the Noea guide) is the right design: a planet has an **integrity** value; if a sufficiently powerful attack's effectivePL exceeds the planet's integrity, the planet is destroyed outright (leaving collectible debris, killing anyone present, awarding an Evil alignment shift). This is a clean single-number threshold that maps onto the canon power-level system directly: a planet's integrity is a PL value, and the check is the same combat-ratio formula.

11.3 Star Destruction Cinematic

Noea v1.0 documents a full star-destruction sequence: the star brightens, shrinks, collapses, and explodes, visible from extreme range. This is a **Niagara VFX sequence + camera event**, not a simulation — there is no point simulating a star's physics in voxels. The right implementation: a scripted cinematic triggered by the attack landing, with the star's mesh/shader running a state machine (brighten → shrink → collapse → supernova), and a screen-shake + shockwave Niagara effect at the moment of collapse.

12. Content & Saga Structure

The content layer is what makes the game a *Dragon Ball* game rather than a combat sandbox. The canon saga structure (DBZ ch. 1–325 + DBS) provides the story backbone; the Noea mod provides the meta-systems (masters, tournaments, raids) that give the player something to do between story beats.

12.1 The Saga Backbone

Saga	Manga chapters	Power-level range	Boss
Raditz / Saiyan	Ch. 195–241	Farmer 5 → Vegeta 18,000	Vegeta (Oozaru ~180,000)
Namek / Ginyu	Ch. 242–285	Cui 18K → Ginyu 120K	Captain Ginyu
Frieza	Ch. 286–329	Frieza 1st 530K → 100% 120M	Frieza (100%)
Trunks / Androids	Ch. 330–419	Androids 17/18/16	Imperfect / Semi-Perfect Cell
Cell Games	Ch. 420–444	Perfect Cell 900M–1.2B → SSJ2 Gohan 1.8B	Perfect Cell
Babidi / Majin Buu	Ch. 445–486	Majin Vegeta → Fat Buu → Super Buu	Super Buu (base)
Fusion / Kid Buu	Ch. 487–519	Buuutenks → Buuhan → Ultimate Gohan 55B	Kid Buu (base)
DBS: God tier	DBS manga	SSG → SSB → UI/UE → Beast	Beerus, Golden Frieza, Broly, Jiren

12.2 Masters and Training Minigames

Five NPC Masters, each teaching one training minigame (Rhythm/Mr. Popo, Control/Krillin, Shadow Boxing/Gohan, Precision/Trunks, Gravity/Vegeta). Rewards scale with the player's current power up to a per-session cap — the single biggest deliberate TP source. This is a directly portable system; it solves the problem of "how does the player grind TP without it feeling like a grind" by making the minigames themselves the gameplay.

12.3 Tournament Bracket Mode

Noea v1.0's bracket mode (per the Noea guide) is the model: an Announcer NPC opens a bracket; survival-mode players create a single-elimination tournament, fighters auto-filled with NPCs or hand-picked. Difficulty scales **opposing NPC health, not raw damage**, to stop one-shot outcomes at matched power (Easy 2× HP, Normal 3×, Hard 5×, Extreme 10×, plus a "story power" mode). This is the right way to handle PvE difficulty in a power-level-scaled game — scaling damage would make every fight a coin flip, while scaling HP gives the player time to express skill.

12.4 Custom Raid Bosses

Noea v1.0's Legendary Core raids (Broly, Janemba, Beerus, Noa) are the model for endgame content. Key documented mechanics worth porting:

- **Motion-capture-referenced attack animations** that adapt in real time to the attacking party's stats, rather than fixed damage numbers.
- **Boss-specific mechanics** — area "ring" zones to vacate, super-dash gap-closer, teleport-behind heavy "flow state" attack.
- **Multiple phases** (documented "phase two" power-up on Broly) — the boss transforms at HP thresholds, requiring the player to have enough forms banked to match.
- **Out-of-raid healing disabled inside arenas** — forces reliance on teammates in party raids; shared teammate-health HUD.
- **Accessibility options** to reduce raid VFX and disable screen shake for lower-end PCs and motion-sensitive players.

13. Build Roadmap — Phased Plan

A phased plan keeps scope under control and produces a playable artifact at every milestone. The research on how Noea itself was built (solo modder, public alpha → beta → v1.0 across roughly a year) and how fighting games are typically scoped (UE5 tutorial series, AAA production cycles) suggests four phases.

13.1 Phase 1 — Combat Prototype (Weeks 1-8)

One character (Goku), one arena (rocky field), the full core combat kit. Goal: prove the Sparking! ZERO feel is achievable before any RPG layer is added.

Deliverable	Done when
Character controller	Walk, dash, fly, ascend, descend all feel responsive; camera tracks 3D arena
Core moveset	Rush Chain (4-hit), Smash Attack (charge+release), Ki Blast, Ki Charge, Throw
Movement kit	Step, Dragon Dash, Z-Burst Dash, Quick Ascend/Descend
Counter ladder	All 5 counters (Super Perception, Z, Revenge, Super, Evasion) with correct windows
Sparking! Mode	Gauge fill, activation, Ultimate Blast, Ki-cost-free actions
Two-character local PvP	Goku vs Goku, first-to-3 matches, no online

13.2 Phase 2 — Vertical Slice (Weeks 9-20)

Three characters (Goku, Vegeta, Piccolo), one full saga (Saiyan saga: Raditz → Nappa → Vegeta), the RPG layer wired in. Goal: prove the merge works — that the RPG grind doesn't break the combat feel.

Deliverable	Done when
RPG layer	Races (Saiyan + 1), six-stat sheet, TP, Power Release, mastery curve wired
Transformation system	SSJ unlock + mastery; Ki-upkeep; mid-fight transform via Skill Count
Canon PL data	Saiyan saga characters in the DB with source-tier tags; combat-ratio damage live
Saga content	Raditz → Nappa → Vegeta story missions; Mr. Popo training minigame
Destructible terrain	Voxel arena; Kamehameha carves terrain; planet-integrity check
Online PvP (rollback)	Goku vs Goku vs Vegeta vs Piccolo, 1v1, rollback netcode

13.3 Phase 3 — Beta (Weeks 21-44)

Full DBZ roster (Saiyan → Buu saga), all six races, all sagas, full transformation ladders, tournament mode, first raid boss. Goal: content-complete enough to ship early access.

13.4 Phase 4 — v1.0 (Weeks 45-60)

DBS god-tier content (UI, UE, Beast, Rosé), space travel, full planet system, all raid bosses, tournament bracket mode, accessibility polish. Goal: the full merged vision.

13.5 Effort and Team-Size Estimate

Phase	Calendar	Team size	Note
1. Prototype	~2 months	1-2 (programmer + animator)	Solo-moddable scope
2. Vertical slice	~3 months	3-5 (+ designer, FX artist, writer)	First hire: combat animator
3. Beta	~6 months	6-10 (+ content designers, netcode specialist)	Rollback specialist is critical hire
4. v1.0	~4 months	8-12 (+ DBS content, space/planet systems)	By then, accessibility pass

These are rough order-of-magnitude estimates informed by Noea's actual solo-dev timeline (one year across v0.3 → v1.0) and typical indie fighting-game scope (Project L, TFH, etc.). A solo developer can reach Phase 1; reaching Phase 3+ realistically

14. Risks, Legal, and Caveats

14.1 Trademark and Legal

Dragon Ball, Dragon Ball Z, Dragon Ball Super, Dragon Ball: Sparking! ZERO, and all character names (Goku, Vegeta, etc.) are trademarks of Bird Studio / Shueisha / Toei Animation / Bandai Namco. A commercial game using these names and likenesses requires a license from Bandai Namco (the interactive rights holder). The Dragon Block Noea / DragonMineZ mods operate in a legally gray fan-work space and are unaffiliated with the rights holders. This document is a research/design blueprint, not legal clearance to ship a Dragon Ball game commercially.

14.2 The "Original IP" Alternative

The cleanest legal path is to build the game with an **original IP inspired by Dragon Ball** — original character names, original planet names, but the same combat feel, the same RPG progression, and the same power-level architecture. The dragon-power.js prototype already in this workspace takes exactly this approach ("Dragon Ball-inspired" with original assets). The canon power-level research transfers directly (it is a numerical-design pattern, not trademarked); the Sparking! ZERO combat mechanics transfer (game mechanics are not copyrightable); only the names and likenesses need to be original.

14.3 Scope-Creep Warnings

- **Do not build 164 characters** before the core loop feels right. Sparking! ZERO shipped 164 because the core kit was shared; building the roster before the kit is the classic fighting-game scope-creep trap.
- **Do not build the space/planet system before combat is fun.** Noea added space in v0.75, well after the combat and progression were proven. Reversing this order produces a pretty sandbox with nothing to do in it.
- **Do not skip rollback.** Shipping a PvP fighting game on delay-based netcode in 2026 is a launch-killer — the fighting-game community will reject it within a week of release, regardless of how good the combat is.

14.4 The "Sparking Clone vs Original" Line

The merged game should be a **Sparking! ZERO-style** arena fighter, not a Sparking! ZERO *clone*. The difference: take the design principles (movement as backbone, Ki as universal economy, layered counters, Sparking!-as-climax, shared core kit) and the RPG inheritance (races, six-stat sheet, TP, mastery), but build original movesets, original transformation cinematics, original characters. The goal is a game that *plays like* the best of both sources without *being* either of them.

15. Bibliography — Research Sources

This blueprint synthesizes four research inputs and supplementary web sources.

15.1 Primary Source PDFs (supplied / built in this workspace)

- **Dragon Ball: Sparking! ZERO — Combat & Controls Guide** (7 pp) — compiled from Bandai Namco Europe's beginner guide, the combo/features page, the NEO DLC announcement, and TheGamer / NoobFeed / GamingBolt / Game8 breakdowns. The source for all combat mechanics.
- **Dragon Block Noea — Deep Research Guide** (9 pp) — documents DragonMineZ (Forge 1.20.1 base mod, 538K+ CurseForge downloads) and the Dragon Block Noea add-on (BuD-eR / ButterJaffa). The source for all RPG progression systems.
- **Dragon Ball Z — Fact-Checked Power Levels** (12 pp) — every PL cross-referenced against Kanzenshuu, Daizenshuu 7, V-Jump, manga chapter readings, and the Kakarot game. The source for all canon power-level data and the source-tier tagging system.
- **Dragon Block C — Deep Research Guide** (14 pp) — the companion research on the older JinGames Java 1.7.10 mod; its canon tables remain valid, its mod-identification was superseded by the Noea guide.

15.2 Canon Power-Level Sources (cited in the fact-checked guide)

- **Kanzenshuu** — the Dragon Ball fan-research community of record; manga compilation readings and the "Over 8,000" (not 9,000) correction.
- **Daizenshuu 7** — Shueisha's official databook; the source for SSJ ×50, SSJ2 ×100, SSJ3 ×400, and Goku SSJ 150M / Frieza 100% 120M.
- **V-Jump magazine** — the source for the SSJ3 Goku = 24 billion promo value.
- **Dragon Ball manga** (ch. 195-519) — on-panel scouter readings: Raditz 1,500, Vegeta 18,000, Frieza 1st 530,000, Ginyu 120,000, Gohan enraged 1,307.

15.3 Game-Design and Engineering Sources (web research)

- **Ars Technica** — "Explaining how fighting games use delay-based and rollback netcode" (2019). The source for the rollback vs delay analysis in Chapter 10.
- **infil.net — The Fighting Game Glossary** — the canonical reference for rollback, delay-based, and fighting-game terminology.
- **CNET / Rolling Stone / Hardcore Gamer** — Sparking! ZERO reviews establishing the "simple to learn, hard to master" and "theatrical look" consensus.
- **GameFAQs / Steam community** — player-base analysis ("Ki management is the name of the game", "every character is a setup archetype") that informs the design-philosophy chapter.
- **gamedev.net / mocaponline.com / gamedev.stackexchange.com** — fighting-game state machine design, blend-tree vs FSM, and patterns for 200+ state systems.
- **Universal Fighting Engine (UFE) — Unity forum discussions** — frame-data and move-definition schema reference.
- **GitHub: Marching-Cubes-On-The-GPU / benwindley.github.io** — GPU compute-shader marching-cubes for destructible voxel terrain.
- **DragonMineZ CurseForge / Modrinth / GitHub** — the base mod's own documentation (GeckoLib 4.8.3+, TerraBlender, six-stat sheet, TP, mastery).
- **Bandai Namco Europe — Sparking! ZERO Beginner's Guide, Combos and Features, NEO DLC announcement** — the official source for all combat mechanics in Chapter 3 and 4.

- **Bandai Namco — Dragon Ball Xenoverse 3 Battle System Guide** — reference for the transformation-as-resource design and the 4-Super/1-Ultimate/1-Assist loadout structure.

— *End of Blueprint* —